

Lesson 3 — Acceleration, Friction And Drag

One lesson: 2 hours 30 minutes · keep this sheet

Name

Class

Date

Cheat sheet

The six steps, in this order

```

// 1. turn
ship.angle += TURN_SPEED * dt * dir;

// 2. build the acceleration from zero
let a = new Vec2(0, 0);
if (thrusting) {
    a = a.add(Vec2.fromAngle(ship.angle)
                .scale(THRUST));
}

// 3. acceleration → velocity
ship.vel = ship.vel.add(a.scale(dt));

// 4. drag, PER SECOND
ship.vel = ship.vel
    .scale(Math.exp(-DRAG * dt));

// 5. limit the LENGTH
if (ship.vel.length() > MAX_SPEED) {
    ship.vel = ship.vel.normalise()
        .scale(MAX_SPEED);
}

// 6. velocity → position (always last)
ship.pos = ship.pos.add(ship.vel.scale(dt));
    
```

The bug that looks fine

```

vel = vel.scale(0.98);    // per FRAME —
wrong
    
```

fps	speed left after 1 s
30	55%
60	30%
144	5.5%

Same code, three machines, three games. Use:

```

vel = vel.scale(Math.exp(-DRAG * dt));
    
```

Drag vs friction

Drag multiplies → takes a *share* → never quite stops → air, water, ships.

Friction subtracts → takes a *fixed amount* → really stops → ground.

```

const loss = FRICTION * dt;
if (vel.length() <= loss) vel = Vec2.zero();
else vel =
    vel.sub(vel.normalise().scale(loss));
    
```

Without that `if`, the ship drives slowly **backwards** for ever.

The numbers are linked

```

settled speed = THRUST / DRAG
half-life      = 0.693 / DRAG
    
```

If `THRUST/DRAG` is below `MAX_SPEED`, the speed limit never fires at all.

Four presets, same code

	THRUST	DRAG	MAX
spaceship	420	0.3	500
car	900	2.5	380
ice	300	0.15	450
snappy	4000	14	320

Words

Acceleration — change in velocity, px/s per second.

Terminal velocity — where push and drag cancel.

Frame-rate dependent — behaves differently on a faster machine. A bug.

Section A — Recall

A1. Name the three levels of “how things move”, in order, and say what each feels like. [3 marks]

A2. Why is `vel = vel.scale(0.98)` a bug, given that no `dt` appears to be missing? [2 marks]

A3. Give two differences between *drag* and *friction*, and say which you would use for a character walking on stone. [3 marks]

A4. Why must a speed limit be applied to the velocity's *length* rather than to `x` and `y` separately? [2 marks]

A5. What is *terminal velocity*, and what two numbers decide it? [2 marks]

A6. Why must the position update come *after* the velocity update, and what is the symptom if it does not? [2 marks]

A7. What is the one line you must never leave out of subtractive friction, and what happens without it? [2 marks]

Section B — Predict the output

B1. `THRUST` is 1200 and `DRAG` is 4. The player holds thrust for a long time. What speed does the ship settle at? If `MAX_SPEED` is 500, does the clamp ever fire? [4 marks]

B2. `DRAG` is 2. Roughly how long does the ship take to lose half its speed? And three quarters of it? [3 marks]

B3. A ship has velocity `(100, 0)`, `THRUST 600` pointing straight down, `DRAW 0`, and `dt` is `0.1`. Give the [4 marks]
velocity and position change after one frame. Starting position `(0, 0)`.

B4. Velocity is `(3, 4)` and subtractive friction removes 6 units this frame. With the zero test, what is [3 marks]
the new velocity? Without it?

B5. One ship runs at 60 fps and one at 20 fps, both using `vel *= 0.9` per frame. After one second, [3 marks]
what fraction of the starting speed does each have left? ($0.9^{60} \approx 0.0018$, $0.9^{20} \approx 0.12$.)

Section C — Find and fix the bug

C1. The ship speeds up for ever and leaves the screen in two seconds. What is missing? [3 marks]

```
if (keys["ArrowUp"]) {  
    ship.vel = ship.vel.add(nose.scale(THRUST * dt));  
}  
ship.pos = ship.pos.add(ship.vel.scale(dt));
```

C2. Diagonal flight is faster than straight flight, even though `MAX_SPEED` is set. Name the bug and write [4 marks]
the correct version.

```
if (ship.vel.x > MAX_SPEED) { ship.vel.x = MAX_SPEED; }  
if (ship.vel.y > MAX_SPEED) { ship.vel.y = MAX_SPEED; }
```

C3. The ship coasts to a near-stop and then creeps very slowly in the wrong direction and never stops. [4 marks]
Why?

```
const loss = FRICTION * dt;  
ship.vel = ship.vel.sub(ship.vel.normalise().scale(loss));
```

C4. The controls feel very slightly dead, as if there is a lag the author cannot measure. The physics is otherwise correct. What is wrong with this order? [4 marks]

```
ship.pos = ship.pos.add(ship.vel.scale(dt));  
ship.vel = ship.vel.add(acceleration.scale(dt));  
ship.vel = ship.vel.scale(Math.exp(-DRAG * dt));
```

C5. With a very low frame rate, the ship suddenly flies *backwards*. `DRAG` is 8 and `dt` reached 0.2. Explain, and name the fix. [3 marks]

```
ship.vel = ship.vel.scale(1 - DRAG * dt);
```

Section D — Build it

Start from your lesson 2 ship.

- ☐ **Checkpoint 1.** Replace “set the velocity” with “add to the velocity”. The ship will now fly off the screen and never come back. That is correct, and it is progress.
- ☐ **Checkpoint 2.** Add drag with `Math.exp(-DRAG * dt)`. Print the speed on screen while you fly — you will need that number for the rest of the lesson.
- ☐ **Checkpoint 3.** Add a speed limit on the *length*. Then check your maths: hold thrust until the speed settles and confirm it equals `THRUST / DRAG`, not `MAX_SPEED`.
- ☐ **Checkpoint 4.** Add four sliders (plain `<input type="range">` is fine) for `THRUST`, `DRAG`, `MAX_SPEED` and `TURN_SPEED`, showing the live value of each.
- ☐ **Checkpoint 5.** Add the four presets on keys `1–4`. Fly each one for thirty seconds before moving on.
- ☐ **Checkpoint 6.** Make a deliberately broken version alongside, using `* 0.98` per frame, and a control that drops your loop to 20 fps. Fly both at both frame rates and write down what you observe.
- ☐ **Checkpoint 7.** Tune your own preset. Write the four numbers on paper, with one sentence saying what sort of game it is for.
- ☐ **Checkpoint 8.** Swap seats and fly your neighbour's ship. Then argue about it — politely, with reasons.

HOW TO DROP YOUR FRAME RATE ON PURPOSE

Do not try to make the computer slow. Call your own update from a `setInterval(step, 50)` instead of `requestAnimationFrame`, with `dt` fixed at 0.05. That gives you an honest 20 fps with no guesswork.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning. Name what your idea costs.

E1. Invent a **procedure** for tuning four sliders from scratch — an order, with the question you are answering at each step. Which of the four can you judge without having settled the other three?

E2. The "snappy" preset is nearly the beginner version. Is the difference worth the extra code? Say how you would **find out** rather than argue: what would you measure, or who would you ask, and what exactly would you ask them?

E3. Design a car that grips going forwards and slides going sideways. Your `Vec2` already has the tool that splits the velocity into those two parts. What would you do to each half, and what would the car then feel like?

E4. The hard one. An "ice" power-up must change all four numbers over half a second, hold them for five seconds, then change them back. What does that machinery look like, and where in your program does it live? Would you reuse it for anything else?

Stretch goals

1. Replace the drag slider with a "seconds to lose half your speed" slider.
2. A dash: a one-off push, a cooldown, and no drag while it lasts.
3. Wind — one extra line in the acceleration step. Then make it vary across the screen.
4. Separate grip along and across the ship (E3, for real). Four lines, biggest

change to feel in the whole level.

5. Measure the settled speed and compare it with `THRUST / DRAG`. Report both numbers.